

Contents

1 - Tokenized bond — register + issue (T0/T1) 2

2 - Tokenized bond — free transfer (T2) 3

3 - Tokenized bond — register payout key (T2b) 4

4 - Tokenized bond — scheduled transformation (T3) 5

5 - Tokenized bond — deadline graduation (T4) 6

1 - Tokenized bond — register + issue (T0/T1)

One transaction registers the CIP-113 instance and mints the first tokens to the beneficiary, publishing the bond’s terms as a CIP-68 reference datum. Registering and minting can share a transaction because the framework lets the first mint prove against the registry node as it is being created.

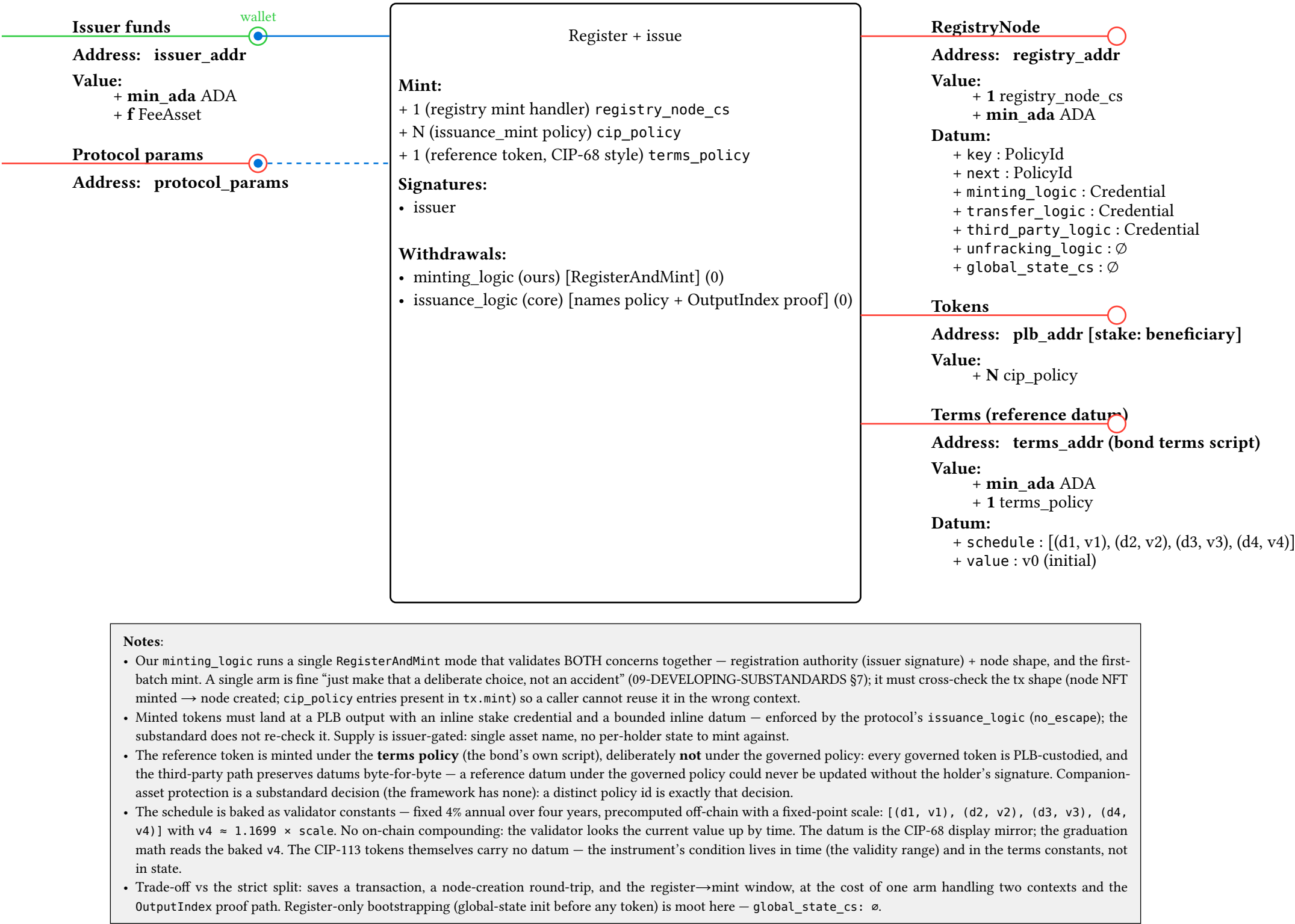
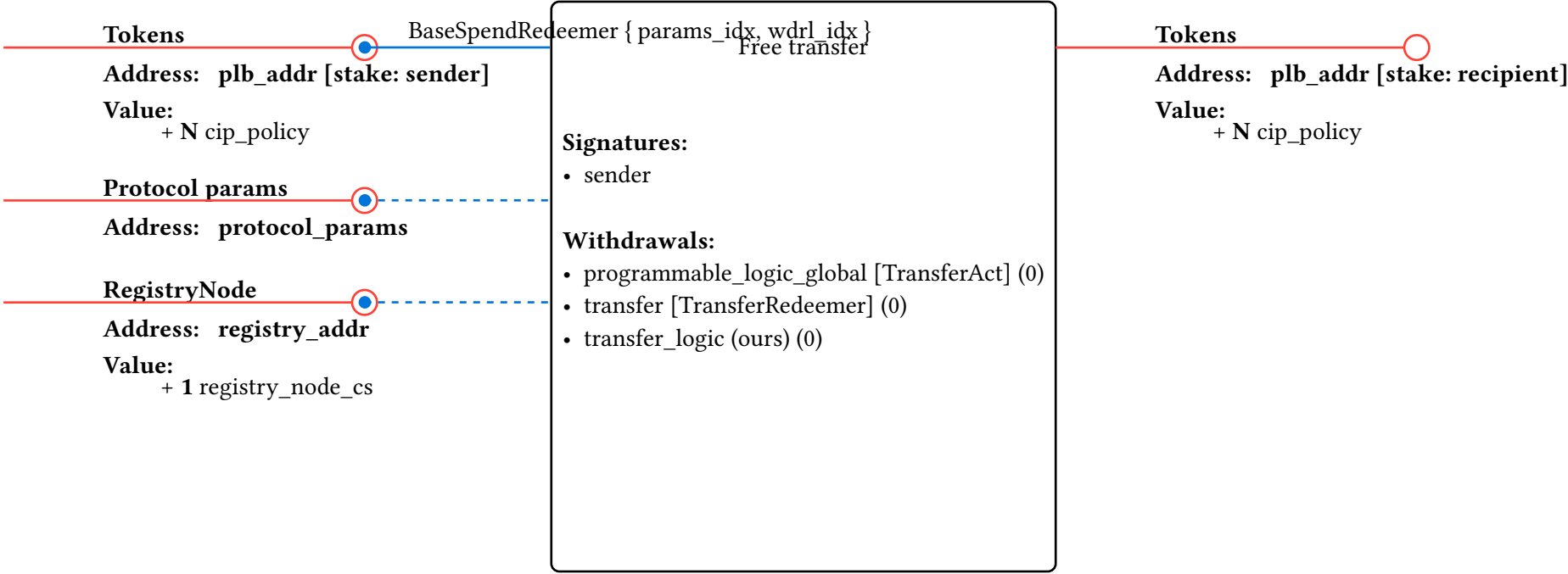


Figure 1: Atomic register + first issue

2 - Tokenized bond — free transfer (T2)

The owner can transfer the token freely at any point in its life — before and after the deadlines. A transfer never gates who may hold or send it.

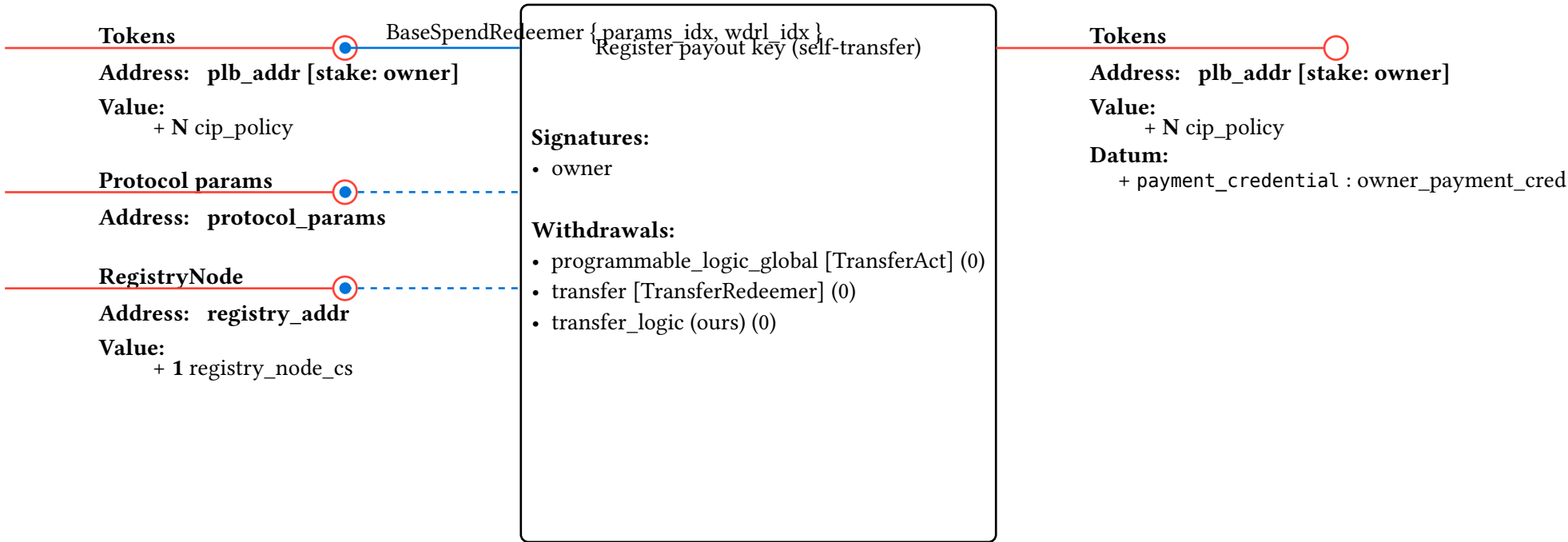


- Notes:**
- Dispatch chain: PLB requires the dispatcher → dispatcher (TransferAct) requires transfer → transfer resolves the registry node and requires our transfer logic (09-DEVELOPING-SUBSTANDARDS §4, §6). A transfer needs **three** script withdrawals, not two — without the dispatcher the tx fails at programmable_logic_base before the transfer validator ever runs.
 - The PLB spend redeemer is BaseSpendRedeemer { params_idx, wdrl_idx } — one record for every PLB input in the tx; it carries **no action arm**. The path is selected by the dispatcher's redeemer (TransferAct), never by the base redeemer.
 - Withdrawals are listed here by role; the ledger presents them in its canonical order (scripts before vkeys, ascending) — builders derive every wdrl_idx from the sorted set (09-DEVELOPING-SUBSTANDARDS §10).
 - P1 at every point of the lifecycle — before and after the deadlines: the permissive transfer_logic (Q-RULE-3) never gates who may hold or send, so the token is DEX/venue-compatible.
 - Ownership rides the inline stake credential of the token UTxO — it moves with the token at every transfer, and it is what the graduation binds the payout to (T4).

Figure 2: Free transfer before or after the deadlines

3 - Tokenized bond — register payout key (T2b)

A special case of the transfer: the owner spends the token to themselves and writes a payment credential into its datum. This is the opt-in that lets someone else (typically the issuer) graduate the token later without being able to redirect the payout — only the owner, by signing this transfer, can set it. An owner who intends to sign their own graduation (T4) never needs this.



Notes:

- Same path as an ordinary transfer (T2): `programmable_logic_global [TransferAct] → transfer → transfer_logic`, owner-signed. The **only** difference is the output datum — the token returns to the same `plb_addr [stake: owner]`.
- The datum commits the owner's payout `payment_credential`. Because the transfer is owner-signed (`authorised_stake_cred`), only the owner can set it — that is what makes it trustworthy at graduation.
- The transfer path bounds the output datum (`max_inline_datum_bytes`) and preserves the seizable output shape (inline stake credential, no reference script), so the commitment does not freeze the token.
- In the third-party graduation path (T4) this datum is preserved byte-for-byte, so a third party can complete the graduation but never redirect the payout.
- Optional and re-settable: the owner may skip it (and sign the graduation directly), or overwrite it with a later self-transfer.

Figure 3: Register payout key via owner-signed self-transfer

4 - Tokenized bond — scheduled transformation (T3)

At each deadline the bond’s value steps up 4%. A transaction updates the terms datum to the schedule’s current value. No signature is required, so anyone can submit it — usually the issuer, but the holder can force it too.

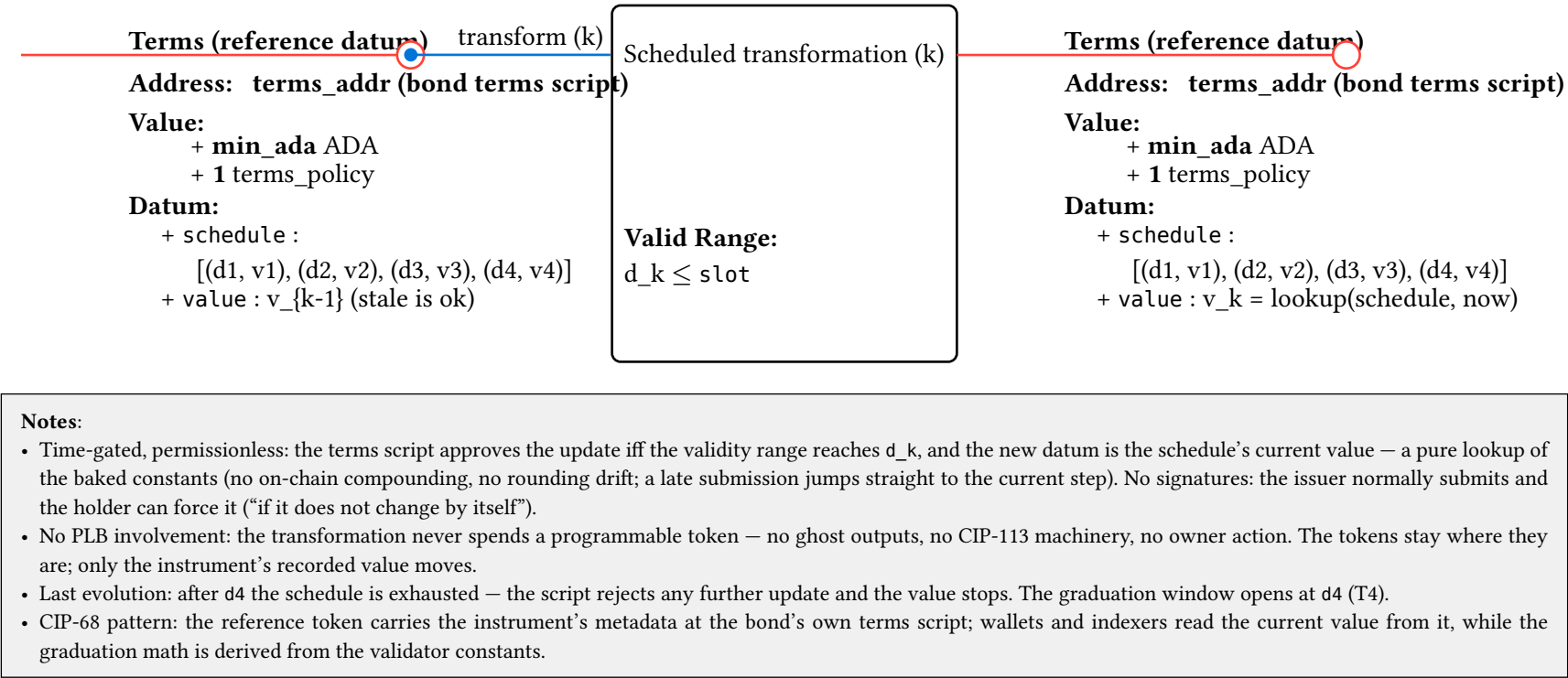
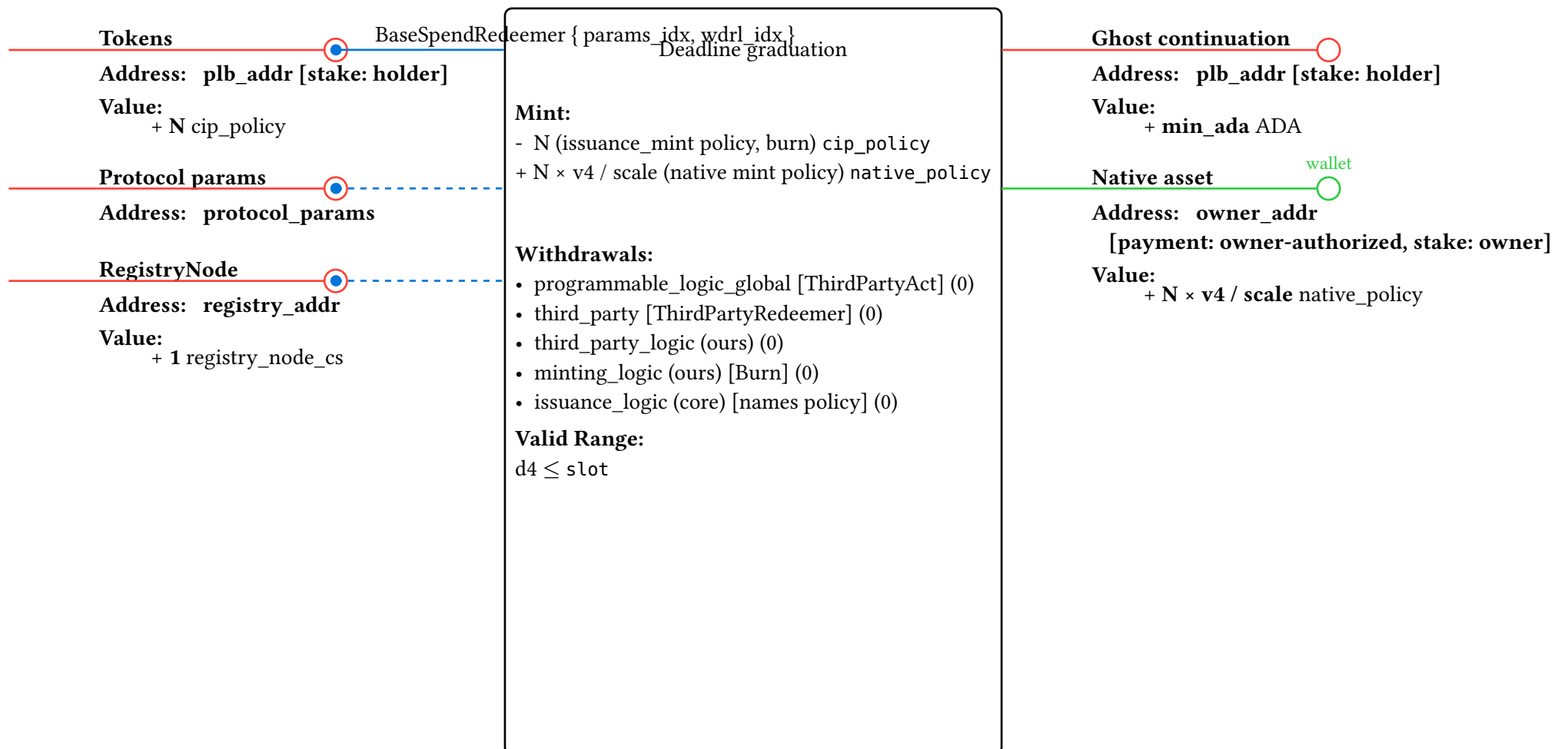


Figure 4: Scheduled value transformation (+4%, holder-passive)

5 - Tokenized bond — deadline graduation (T4)

From $d4$ on the tokens **may** convert into the corresponding native asset at the schedule’s final value — graduation is *opt-in*, never forced. It is allowed only when the payout destination is owner-authorized: the owner either signs the graduation and names the address, or has pre-committed a payment credential into the token’s datum on an earlier owner-signed transfer (T2b). Without one the transaction is rejected — there is no fallback, because a native asset sent to the wrong spending key is unrecoverable. A holder who never opts in simply keeps the (now non-transforming) token.



Notes:

- A burn fires `issuance_mint` with a negative quantity — the **same two** issuance withdraw-0s as a mint (`protocol issuance_logic + ours`) — plus the full spend chain of whichever action releases the tokens (09-DEVELOPING-SUBSTANDARDS §6).
- Event gate (Q-RULE-1): there is no separate rule script — the deadline check lives inside the substandard logic that already runs (the `third_party_logic / transfer_logic` withdraw-0 on the spend side and the `minting_logic` withdraw-0 on the burn side). It approves iff the validity range reaches $d4$ — time-driven, no oracle; the event condition is the tx’s own validity range.
- Authorization (Q-GRAD-2, this instrument): the $d4$ event gate is necessary but not sufficient — the burn also needs an **owner-authorized payout destination**. That is **either** the owner’s signature (owner path, which names the destination in the tx) **or** a payment credential the owner pre-committed to the token datum (which a third party can then complete without redirecting). The permissionless third-party path therefore only works for tokens whose owners opted in beforehand; with no owner signature and no committed destination, the tx is rejected.
- The conversion function (the Burn-mode validation) is shared by both paths: burn shape, full-burn per asset name (whole holdings burn — it makes the per-owner destination attribution exact), destination binding and the scaled native mirror. The native minting policy is signer-agnostic: it approves the mint only against the burn (`mint == burned × v4 / scale`), whichever credential signed it.
- Destination binding: the native asset is a plain token at a normal (`payment_credential`, `stake_credential`) address, but CIP-113 attributes ownership only by the **stake** credential — the payment (spending) key is not something the framework can tie to the owner. So the payment credential must come from the owner: named in an owner-signed graduation, or pre-committed to the token’s inline datum on an owner-signed transfer (T2b, preserved byte-for-byte through the third-party path, so a third party can complete but never redirect it). The stake credential stays bound to the owner’s current credential.
- No safe fallback, so no fallback: absent an owner signature **and** a committed payment credential, the transaction is rejected. Sending a native asset to an unverified spending key is unrecoverable, and graduation is optional — the owner may hold the final-value, non-transforming token indefinitely rather than convert it.
- Native mint: exactly `burned × v4 / scale`, regardless of which credential signed the burn — the native policy approves the mint only because it is backed by the governed burn of the same name at the schedule’s final value (“no burn CIP, no mint”, scaled). $v4$ is the baked constant ($\approx 1.1699 \times \text{scale}$): the only on-chain arithmetic is this final multiply. The mint is deliberately not 1:1 quantity-conserved — the scaled value rules that out — so it is governed by the native policy and the substandard’s `minting_logic` instead.
- Third-party path mechanics: the PLB spend’s paired continuation must preserve address, datum and reference script byte-for-byte (lovelace is **ratcheted** — output $\geq$ input, not conserved) — the **ghost** output (one-time per spent UTxO, reclaimable by the owner via a transfer-path spend). In the issuer’s case the ghost remains; on the owner path (transfer path + the same Burn mode) nothing remains — the cheaper shape.
- Base-layer guarantee: a transaction that spends a registry node can never mint or burn that node’s own token — graduation is pure issuance, never mixed with a registry reconfiguration.
- Asset identity (Q-GRAD-1): the native asset is a new policy — DEX pools / price history continuity across the flip remains an open global question.

Figure 5: Deadline graduation (opt-in, value-preserving)